



PROCESADORES DE LENGUAJES
Ingeniería Informática
Especialidad de Computación
Tercer curso, segundo cuatrimestre



Departamento de Informática y Análisis Numérico
Escuela Politécnica Superior de Córdoba
Universidad de Córdoba
Curso académico 2022 - 2023
Examen de la primera convocatoria
2 de junio de 2023

Teoría

1. Traducción e interpretación

- Descripción
- Diferencias

2 puntos

2. Formas normales de las gramáticas de contexto libre

- Descripción
- Ejemplos

1 punto

Ejercicios

3. Análisis léxico

- Dada la siguiente expresión regular: $(a + b c)^*$
 - a) Utiliza el Algoritmo de Thompson para construir un AFN equivalente a la expresión regular.
 - b) Utiliza el Algoritmo de Construcción de Subconjuntos para obtener un AFD equivalente al AFN obtenido en el apartado "a".
 - c) Minimiza, si es posible, el AFD obtenido en el apartado anterior.
 - d) Comprueba si el último autómata obtenido reconoce la siguiente cadena: $a b c a$

2 puntos

4. Análisis descendente predictivo

- Considérese la siguiente gramática de contexto libre:

$P = \{$

- 1) $S \rightarrow S D$
- 2) $S \rightarrow D$
- 3) $D \rightarrow \text{typedef struct } \{ C \} id ;$
- 4) $C \rightarrow C L$
- 5) $C \rightarrow L$
- 6) $L \rightarrow T id ;$
- 7) $T \rightarrow int$
- 8) $T \rightarrow double$

$\}$

- Esta gramática permite generar declaraciones de tipos de datos usando estructuras como, por ejemplo,

`typedef struct { double x; double y; } Punto ;`

donde "x", "y" y "Punto" son identificadores.

- a) Elimina la recursividad por la izquierda y factoriza la gramática por la izquierda.

- b) A partir de la gramática obtenida en el apartado "a":
- Construye los conjuntos "primero" y "siguiente" de los símbolos no terminales.
 - Construye la tabla de análisis descendente predictivo.
 - Utiliza el método recuperación de errores de "nivel de frase" para completar la tabla predictiva.
 - Utiliza la tabla predictiva para realizar un análisis descendente no recursivo de la siguiente enumeración errónea:
`typedef { { double ; ; y } Punto`

2 puntos

5. Análisis sintáctico ascendente SLR

- Considérese la siguiente gramática de contexto libre:

$$P = \{$$

- 1) $S \rightarrow S D$
- 2) $S \rightarrow \varepsilon$
- 3) $D \rightarrow L : T ;$
- 4) $L \rightarrow \text{id } L'$
- 5) $L' \rightarrow , L$
- 6) $L' \rightarrow \varepsilon$
- 7) $T \rightarrow \text{int}$
- 8) $T \rightarrow \text{float}$

$$\}$$

que permite generar declaraciones de variables como las siguientes:

`a, b, c: integer;`
`x, y: float;`

donde *a, b, c, x e y* son *identificadores*

- a) Construye los conjuntos "primero" y "siguiente" de los símbolos no terminales.
- b) Construye la colección canónica de LR(0)-elementos del análisis SLR.
- c) Dibuja el autómata que reconoce los prefijos viables.
- d) Construye la tabla de análisis sintáctico SLR: partes acción e ir_a.
- e) Utiliza el método recuperación de errores de "nivel de frase" para completar la tabla de análisis SLR.
- f) Utiliza la tabla de análisis SLR-canónico para analizar la siguiente declaración errónea:
`, b c integer integer`

2 puntos

6. Diseño de una gramática de contexto libre

- Diseña una gramática que permita generar "algunas" declaraciones de variables de tipo *array* el lenguaje de programación Pascal, como, por ejemplo:

`temperatura: array [-20 .. 100] of real;`
`datos: array [1 .. 5] of integer := [90, 18, -12, 37, 10];`

donde *temperatura* y *datos* son *identificadores*.

1 punto